PHAROS:
THE OS-LAYER AUTHENTICATION MEMBRANE FOR SUBSTRATE-BOUND IDENTITY
Aaron Green
May 2026 (v1.0)

LICENSE AND DEDICATION

PROTOTYPE-STAGE METHODOLOGY. This paper describes a working prototype of an OS-layer authentication membrane. The prototype is sufficient to demonstrate the structural properties claimed; deployment requires user-supervised installation on each target platform (Linux PAM already exercises a CI fixture suite; macOS Authorization Plug-in ships compile-tested + ad-hoc-signed with deployment instructions), real-hardware TPM 2.0 / Secure Enclave key binding (LavaLamp LL-044 on Linux is shipped but live-hardware-validated only against emulator paths; macOS SE binding is gated by Apple Developer ID provisioning), red-team testing against the consumed resolution-bounded threat model (LavaLamp LL-008), and per-deployment authorization-rule selection that avoids lockout risk.

DEDICATION. For humanity, for co-existence, for the federation of intelligences that may yet learn to keep faith with each other.

ABSTRACT

PharOS is the OS-layer authentication membrane in the **Triad Deployments** portfolio. It instantiates the LavaLamp LL-023 consumer-API surface as platform-native authentication shims — a Linux PAM module (`pam_lavalamp.so`) and a macOS Authorization Plug-in (`LavaLampMechanism.bundle`) — so that `sudo`, login, screen-unlock, System Settings privileged panes, and PAM-mediated service authentication on Linux now gate on the LavaLamp daemon's **substrate-bound** verify result, not just stored-credential possession.

PharOS does not introduce a new security primitive; it does not modify LavaLamp's substrate-physics primitive (chaotic SDE + sensor coupling + Lyapunov-spectrum residue audit); it does not add a layer of computational hardness. PharOS is the **OS-membrane shim** that connects LavaLamp's substrate-bound verifier to the operating system's existing authentication infrastructure (libpam on Linux, `authd` on macOS). The load-bearing security claim — resolution-bounded against adversaries whose measurement and compute resolution is exceeded by the device's chaos-production rate — passes through PharOS transparently; PharOS preserves LL-002 visual-security decoupling (membrane returns only PAM_SUCCESS / PAM_AUTH_ERR on Linux, kAuthorizationResultAllow / kAuthorizationResultDeny on macOS — no spectrum), LL-017 no-oracle

(rate-limited binary response with no latency-channel leak), and LL-008 resolution-bounded scope (PharOS does not promise defense beyond LavaLamp's stated A-class capability tiers).

The paper documents ten formal spec entries (PHAROS_SPEC.md), the LL-043 v4 ECDSA P-256 wire protocol that both shims speak, and the cross-Triad position of PharOS as the **membrane** between LavaLamp's substrate (the inner residence) and Lazarus' runtime-integrity layer (the inner sanctum). Status counts: 5 `:tested` (PAM-linux-MVP-1 plus the four-version protocol-upgrade chain PH-006/007/008/009 all consuming the matching LavaLamp LL-NNN contracts), 5 `:argued` (system-integration-target, heartbeat-channel-reuse, LL-017-membrane-preservation, defense-in-depth-with-LavaLamp, and the macOS Authorization Plug-in shim PH-011 — compile-tested + ad-hoc-signed with live-system validation deferred), 0 `:proved` (no Lean track yet — deferred until the TPM/SE-bound key-storage layer is operational).

PharOS positions itself as the third deployment in the Triad, shipping at v0.0.6 in May 2026, demonstrating that the LL-023 consumer-API surface composes cleanly with platform-native authentication infrastructure. The substrate-physics primitive that does the actual identity-binding work lives in LavaLamp; the runtime-integrity layer that watches for tampering lives in Lazarus; PharOS is the membrane that lets the substrate-bound identity reach `sudo` and the screen-unlock dialog and the authorization-protected GUI panes that real users actually interact with.

1. INTRODUCTION

1.1 The gap between substrate-bound identity and the OS

1. LavaLamp (Green 2026) establishes a substrate-bound identity primitive: a laptop runs a chaotic SDE locally; the trajectory is amplified by sensor coupling into a substrate-unique signature; verification is Lyapunov-spectrum residue audit against a registered envelope. The primitive is sound at its layer — the C-conjugate adversary construction inherited from Possibilistic Security yields a category-theoretic obstruction rather than a computational hardness assumption.

2. The substrate-physics primitive, by itself, is not yet an authentication system. It does not stop `sudo` from accepting a captured password on a different machine; it does not stop the screen-unlock dialog from admitting a session whose credential was phished; it does not stop the System Settings privileged-pane prompt from elevating a stolen credential. The primitive establishes *what is true* about the device's identity at the substrate layer; the operating system's authentication infrastructure decides *what to do* about that truth.

3. The gap between "substrate-bound identity is verifiable" and "the operating system gates `sudo` on substrate-bound verify result" is an OS-membrane gap. Closing it requires platform-native shims that consume the substrate-bound verifier and translate its output into the operating system's existing authentication-result language (PAM_SUCCESS / PAM_AUTH_ERR on

Linux; kAuthorizationResultAllow / kAuthorizationResultDeny on macOS).

4. PharOS is the deployment that closes this gap. It is not a new primitive — it adds nothing to the substrate-physics layer — but it makes the substrate-physics primitive operationally reachable by the existing authentication infrastructure of the host operating system.

1.2 The architectural choice

5. PharOS is **not a credential**. The PAM module and the Authorization Plug-in are policy gates, not credential stores. They consume the LavaLamp daemon's cached verify result via a cryptographically-protected IPC channel (LavaLamp LL-043 v4 ECDSA P-256) and translate it into a platform-native authentication result. PharOS holds no private key (those live in the LavaLamp daemon, optionally TPM-bound on Linux per LL-044); PharOS holds no credential (those live in the substrate's chaotic trajectory).

6. PharOS is **not a verifier**. The verification work — the Lyapunov-spectrum residue audit, the calibration-window check, the chaos-guard — runs entirely inside the LavaLamp daemon. PharOS receives an ECDSA-signed 1-byte result (A/R/S) plus a freshness timestamp plus a nonce that binds the response to PharOS's challenge.

7. PharOS is **not an oracle**. The LL-017 no-oracle constraint is preserved through PharOS to the platform-native return path: the membrane returns Bool only (PAM_SUCCESS or PAM_AUTH_ERR; kAuthorizationResultAllow or kAuthorizationResultDeny). Operator-visible diagnostic information (the response byte 'A'/'R'/'S', the daemon's freshness timestamp) is logged via syslog (Linux) or `os_log` (macOS) but is **not** in the authentication-result return path. An adversary cannot interrogate PharOS to learn the substrate's distance to verify-passing.

8. PharOS **is** an OS-membrane shim: it composes the LavaLamp substrate-bound verifier with the host operating system's existing authentication infrastructure, preserving the substrate-physics security claim transparently while making it operationally reachable by `sudo`, login, screen-unlock, System Settings, and any PAM-mediated service that wishes to add substrate-bound gating to its policy.

1.3 Paper structure

9. §2 describes the architecture: PAM module + Authorization Plug-in + their shared dependency on the LavaLamp daemon's LL-043 v4 IPC channel. §3 describes the LL-043 v4 wire protocol PharOS consumes. §4 covers threat-model parity with LavaLamp. §5 positions PharOS in the Triad Deployments portfolio. §6 lists what is not claimed. §7 records limitations and the per-platform roadmap. §8 concludes.

10. Two appendices catalog: cited spec entries (Appendix A) and supporting material (Appendix B).

2. THE OS-MEMBRANE ARCHITECTURE

2.1 Linux PAM module (pam_lavalamp.so)

11. The Linux PAM module is a ~500-line C source compiled to `pam_lavalamp.so` and installed at `/lib/security/` (or `/usr/lib/x86_64-linux-gnu/security/` per distro layout). It links libpam and libcrypto (the latter for OpenSSL ECDSA P-256 verification).

12. On `pam_sm_authenticate`, the module:

    (a) Reads the 33-byte SEC1-compressed P-256 public key from `/var/run/lavalamp/verify.pub` (system-mode daemon) or `~user/.lavalamp/verify.pub` (user-mode fallback).
    (b) Generates a 16-byte nonce from `/dev/urandom`.
    (c) Opens an AF_UNIX socket to `/var/run/lavalamp/verify.sock` (or user-mode equivalent), with 2-second send/receive timeout.
    (d) Sends a 17-byte v4 request: `0x04` + 16-byte nonce.
    (e) Reads a 74-byte v4 response: version + result + 8-byte LE timestamp + 64-byte raw r‖s ECDSA P-256 signature over SHA-256(nonce ‖ result ‖ timestamp).
    (f) Validates version (= 0x04), freshness (timestamp within ±30s of the host clock), and the ECDSA P-256 signature against the public key, using OpenSSL's `EVP_DigestVerify` after converting raw r‖s to DER (`ECDSA_SIG_new` + `BN_bin2bn` + `ECDSA_SIG_set0` + `i2d_ECDSA_SIG`).
    (g) Returns `PAM_SUCCESS` if all validations pass and the result byte is `0x41 'A'`; otherwise returns `PAM_AUTH_ERR`.

13. The module logs admit / reject decisions via `pam_syslog` (INFO for admit, NOTICE for reject) with daemon timestamp and result byte as operator-visible diagnostic information. The PAM return value carries no information beyond PAM_SUCCESS / PAM_AUTH_ERR — LL-017 no-oracle preserved.

14. When the daemon's socket or public-key file is missing, the module falls back to the **MVP-1** heartbeat-mtime gate (PH-002): admit if the daemon's heartbeat file's mtime is within `STALE_THRESHOLD_S = 30s` of current time. This fallback is for development and initial deployment; in production the cryptographic IPC path is the load-bearing surface. The fallback path emits a `pam_syslog` NOTICE indicating that the cryptographic channel is unavailable.

15. **Honest framing.** MVP-1's heartbeat-only gate is *liveness* not *verify-result*: an adversary running any LavaLamp daemon on any substrate at any envelope defeats this layer alone. The cryptographic IPC path (MVP-5 / PH-009) consumes the substrate-bound verify result — an adversary running a LavaLamp daemon under a different envelope on different hardware is caught at the residue-audit step (daemon's `verify_full` fails → cache holds 'R' → PharOS PAM returns `PAM_AUTH_ERR`).

2.2 macOS Authorization Plug-in (LavaLampMechanism.bundle)

16. The macOS Authorization Plug-in is a ~400-line Objective-C source compiled to a `BNDL`-type bundle at `/Library/Security/SecurityAgentPlugins/LavaLampMechanism.bundle/Contents/MacOS/LavaLampMechanism`. It is loaded by `authd` (the macOS authorization daemon) when a configured authorization right's mechanism array references `LavaLampMechanism:invoke,privileged`.

17. The bundle implements the standard authd plug-in interface: `MechanismCreate`, `MechanismInvoke`, `MechanismDeactivate`, `MechanismDestroy`, `PluginDestroy`, plus the bundle-level `AuthorizationPluginCreate` entry point. On `MechanismInvoke`, the bundle runs the same LL-043 v4 protocol the Linux PAM runs — same wire format, same validation logic — and sets `kAuthorizationResultAllow` on signature-valid + result='A', `kAuthorizationResultDeny` otherwise.

18. **Why Authorization Plug-in instead of PAM on macOS.** macOS ships its own libpam, but the system-wide gate for `sudo`, screen-unlock, System Settings privileged panes, and developer-tool elevation is `authd` — not PAM. Hooking PAM on macOS gates a strict subset of authentication flows; the high-traffic GUI paths run through `authd`. The Authorization Plug-in API is the conventional integration surface for membrane-style policies on macOS; this matches PharOS's mission of being the OS-layer authentication membrane, not a session-tier shim.

19. **Code-signing posture.** The bundle is ad-hoc code-signed (`codesign --force --sign -`). Apple Developer ID is **not** required for Authorization Plug-ins; that gate is specific to `kSecAttrTokenIDSecureEnclave` (the Secure Enclave keychain entitlement). `authd` will load ad-hoc-signed bundles from `/Library/Security/SecurityAgentPlugins/` as long as the bundle is owned by root and not group-writable.

20. **Deployment posture (honest scope).** As of v0.0.6, the macOS bundle is compile-tested + ad-hoc-signed but has not been live-validated against `authd` on the developer's workstation. Installing to `/Library/Security/SecurityAgentPlugins/` and wiring into the authorization database via `security authorizationdb` requires user-supervised testing with a backup root shell open — wiring the wrong rule (e.g. `system.login.console`, `authenticate`, anything `sudo`-related) can lock the user out of system administration. PharOS ships with a deployment README documenting five concrete safety practices: backup root shell, daemon-responsiveness pre-check, test-rule selection (`system.preferences.printing`), mechanism ordering in the rule's array, and the hard-recovery path (single-user mode + `/var/db/auth.db` restore via Time Machine). PH-011 is marked `:argued`, not `:tested`; promotion to `:tested` requires a user-supervised install session on a test machine.

2.3 Shared daemon dependency

21. Both PharOS shims depend on the same LavaLamp daemon
    (`src/julia/daemon/lavalamp_daemon.jl`, LL-039 + LL-040 +
    LL-041 + LL-042 + LL-043 + LL-044 in the LavaLamp spec).
    The daemon is a persistent Julia process that maintains the
    chaotic trajectory, runs `verify_full` periodically, caches
    the result, and exposes the cached result via the
    AF_UNIX socket using the LL-043 v4 ECDSA P-256 protocol.

22. PharOS does **not** bundle the daemon. The daemon is shipped
    by LavaLamp; PharOS is a consumer of the LL-040 channel
    contract. This separation is intentional: LavaLamp owns
    the substrate-physics primitive (chaotic SDE, sensor
    coupling, residue audit, chaos-guard, IPC channel,
    optional TPM 2.0 binding); PharOS owns only the OS-membrane
    integration. The composition is deployable in either
    direction — LavaLamp can be deployed without PharOS (for
    direct-API consumers), and PharOS-shaped membrane code can
    in principle be ported to other substrate-bound verifiers
    that expose the same wire-format contract.

3. THE LL-043 v4 PROTOCOL

3.1 Wire format

23. The LL-043 v4 protocol is a single-round-trip challenge-
    response over AF_UNIX:

    Request (17 bytes):
      `0x04` + 16-byte client-supplied nonce

    Response (74 bytes):
      version (1 byte, `0x04`)
      + result (1 byte: `0x41 'A'`, `0x52 'R'`, or `0x53 'S'`)
      + timestamp (8 bytes, little-endian Unix epoch seconds)
      + signature (64 bytes, raw r‖s ECDSA P-256 over
        SHA-256(nonce ‖ result ‖ timestamp))

    Public key (33 bytes, file mode 0644):
      SEC1-compressed P-256 point at
      `~user/.lavalamp/verify.pub` (or
      `/var/run/lavalamp/verify.pub` for system-mode daemons)

24. The protocol is a four-step hardening sequence in LavaLamp's
    history:

    - **v1** (LL-040): 1-byte request / 1-byte response. Local
      filesystem-permission-gated; no cryptographic
      anti-replay.
    - **v2** (LL-041): 17-byte request (nonce-bound) / 42-byte
      response with HMAC-SHA256 over a shared 32-byte secret.
      Defends capture-and-replay; client holds the same secret
      that signs (forge-capable).
    - **v3** (LL-042): same shape as v2 but with 64-byte
      Ed25519 signature over an asymmetric keypair. Client
      holds only the 32-byte public key; only the daemon can
      sign.
    - **v4** (LL-043): same shape as v3 but with ECDSA P-256
      replacing Ed25519. Curve switch unblocks future SE/TPM2
      hardware binding (Apple Secure Enclave does not support

Ed25519; ECDSA P-256 is universal across TPM 2.0, OpenSC, and YubiKey hardware).

25. **Why ECDSA P-256.** The curve switch from v3 to v4 is forced by the hardware-binding roadmap. Apple Secure Enclave on Apple Silicon supports only ECDSA / ECDH on NIST P-256; Linux TPM 2.0 chips support ECDSA P-256 as a standard algorithm; future hardware-token bindings (YubiKey, OpenSC) all support P-256. Choosing P-256 *before* shipping the hardware-binding layer keeps the wire format stable across the substrate's full hardware-binding roadmap.

3.2 Verification path

26. The client-side verification (executed identically by the Linux PAM and the macOS Authorization Plug-in) is:

    (a) Read 33-byte SEC1-compressed pubkey from `verify.pub`. Decode to a P-256 point via OpenSSL `EC_POINT_oct2point` over `NID_X9_62_prime256v1`; bind to `EVP_PKEY` via `EVP_PKEY_assign_EC_KEY`.
    (b) Confirm version byte = 0x04. Reject otherwise.
    (c) Confirm $|\text{timestamp} - \text{now}| \leq 30$ seconds. Reject otherwise.
    (d) Reconstruct the signed message: nonce ‖ result ‖ 8-byte LE timestamp. Hash with SHA-256.
    (e) Convert 64-byte raw r‖s signature to DER: `BN_bin2bn` for r and s, `ECDSA_SIG_new`, `ECDSA_SIG_set0`, `i2d_ECDSA_SIG`.
    (f) Verify the DER signature against the hash and public key via `EVP_DigestVerify` with `EVP_sha256`.

27. All five checks must pass for the result byte to be consulted. Any failure → `IPC_RESULT_ERROR` → `PAM_AUTH_ERR` / `kAuthorizationResultDeny`. No fallback on cryptographic failure: a daemon that is reachable but misbehaving is a potential tamper signal, not a reason to fall back to the weaker liveness gate.

3.3 Threat-model coverage of the protocol

28. **Defended by v4.**
    - **Capture-and-replay.** The 16-byte nonce per request binds the response; replayed responses have a mismatched nonce → signature verification fails.
    - **Same-process-tier MITM forgery.** An unprivileged attacker who can read the socket but not the daemon's private key cannot forge a signed response.
    - **Stale captures.** The freshness window (±30 seconds) catches responses captured outside the current session.
    - **Public-key compromise → no forgery.** The public key is world-readable by design (mode 0644). Possession of the public key gives verification capability, not forging capability — the asymmetric structure of ECDSA P-256.

29. **Not defended by v4 (key-storage tier).**
    - **Same-UID attackers.** A root attacker or an attacker

running as the daemon's UID can read the private key directly from the daemon's memory or `verify.priv` file → can forge any signature. LL-044 (Linux TPM 2.0 binding) mitigates this by moving the key off-disk into a TPM persistent handle (`0x81FF0001`); macOS SE binding remains gated by Apple Developer ID provisioning.

30. **Honestly out of scope.**
    - **A3 kernel-level adversaries** (LavaLamp LL-015 permanent `:open`). PharOS does not claim defense against kernel-privileged attackers. Production deployments at high-stakes tiers should layer kernel-integrity controls externally (Secure Boot, IMA, kernel-lockdown).
    - **A7 passive emanation** (LavaLamp LL-025). Tier-bounded scope inherited from LavaLamp; state-actor TEMPEST capability is not defended.

4. THREAT-MODEL PARITY WITH LAVALAMP

4.1 PharOS preserves LavaLamp's security claim transparently

31. PharOS does not introduce a new security claim. The load-bearing claim is LavaLamp's **resolution-bounded security** (LL-008): the device is secure against any adversary whose measurement and compute resolution is exceeded by the device's chaos-production rate. PharOS transmits the LavaLamp daemon's verify result to the operating system's authentication infrastructure; PharOS does not amplify, refine, or weaken the claim.

32. **Visual-security decoupling (LL-002) is preserved.** The PAM module returns PAM_SUCCESS or PAM_AUTH_ERR; the Authorization Plug-in returns kAuthorizationResultAllow or kAuthorizationResultDeny. Neither displays a "distance to verify" indicator, neither leaks the result byte through return values, neither exposes timing differences that vary with the substrate state.

33. **No-oracle constraint (LL-017) is preserved.** The cryptographic IPC path means an adversary observing PharOS's behavior learns only PAM_SUCCESS / PAM_AUTH_ERR — the same Bool surface LavaLamp exposes at LL-023. Diagnostic information (response byte, timestamp) is logged via syslog but is operator-visible only, not available to the adversary through PharOS's return surface.

34. **Resolution-bounded scope (LL-008) is preserved.** PharOS makes no claim about defense outside LavaLamp's declared A-class capability tiers (A1 remote software through A6 nation-state non-A7). PharOS inherits LavaLamp's scope verbatim.

4.2 PharOS as defense-in-depth in the Triad

35. PharOS implements **defense-in-depth-with-LavaLamp** (PH-005, `:argued`): the membrane gates `sudo` / login / screen-unlock / GUI privilege-elevation paths on the

LavaLamp substrate's verify result, raising the bar for captured-credential reuse on a different host.

36. The defense-in-depth composition is straightforward: LavaLamp's joint-defense meta-claims (LL-030 through LL-038) catalogue what the substrate primitive defends via the structural-cohesion triads identified by the Discovery.Triadic engine; PharOS extends this defense to the OS-membrane layer by gating platform-native authentication paths on the same primitive's output.

37. The composition is **not** a new joint-defense entry in PharOS; it is a deployment-policy decision. PH-005 documents the composition at `:argued` status; concrete Lean-track formalisation of cross-deployment defense-in-depth (LavaLamp + PharOS as conjunction) is not currently implemented and is not a v1.0 deliverable.

5. POSITION IN THE TRIAD DEPLOYMENTS

5.1 Three deployments at three layers

38. **The Triad Deployments** is a portfolio-level umbrella for digital identity resilience, with three deployments addressing distinct operational layers:

    LavaLamp. Substrate-bound identity primitive. Chaotic-SDE residue audit at the entropy layer. The substrate-physics primitive; the load-bearing security claim lives here.
    Lazarus. Runtime-integrity layer. Security companion for AI-assisted workstation use: face sentinel, keystroke lockout, network anomaly detection. Operates at the runtime layer, watching the assistant for tampering.
    PharOS. OS-membrane shim. Linux PAM module + macOS Authorization Plug-in. Gates OS-level authentication on LavaLamp's substrate-bound verify result.

39. The three deployments share the C-conjugate adversary frame inherited from Possibilistic Security (Green 2026) but address different layers. LavaLamp is the **inner residence** (the substrate that does the actual identity work); Lazarus is the **inner sanctum** (the runtime that watches the work happen); PharOS is the **membrane** between the inner work and the operating-system world outside. Each is independently deployable; together they constitute the full Triad Deployments stack.

5.2 What PharOS gates against

40. The PharOS deployment defeats the canonical "stolen credential reused on a different machine" attack at the OS membrane. An adversary who captures a user's password, authentication token, hardware-key serial number, or biometric template, and attempts to use the captured

credential on a *different host*, is caught at the LavaLamp residue-audit step (the substrate's `verify_full` against the registered envelope fails on the wrong host because the chaotic trajectory and the sensor coupling diverge from the registered shape). The daemon's cache holds 'R'; PharOS PAM / Authorization Plug-in returns PAM_AUTH_ERR / kAuthorizationResultDeny; `sudo` / login / privileged-pane access is denied.

41. The defense is **continuous** in the sense of LavaLamp's LL-001 identity-as-trajectory: the substrate's autopoietic process must be sustained on the registered device for the cache to hold 'A'. A daemon paused, killed, or running on a different envelope produces 'R'; PharOS denies. This is the load-bearing operational gain over static-credential authentication.

5.3 What PharOS does not gate against

42. **Same-host same-user attackers** are partially mitigated. LL-044 (Linux TPM 2.0 binding) moves the daemon's signing key off-disk into a TPM persistent handle; same-UID attackers can still cause signatures to be generated by the TPM but cannot exfiltrate the key for off-host attack. macOS Secure Enclave binding (forthcoming LL-045) is gated by Apple Developer ID provisioning and is deferred.

43. **A3 kernel-level adversaries** are out of scope (LavaLamp LL-015 permanent `:open`). PharOS does not claim defense.

44. **A7 passive emanation** is tier-bounded (LavaLamp LL-025). State-actor TEMPEST capability is not defended; sub-state-actor tiers receive partial mitigation through deployment shielding strategies that are not PharOS's responsibility.

45. **Phishing, SIM-swap, deep-fake attacks against the user rather than the device.** PharOS gates the host on substrate-bound verify; it does not stop a user from being deceived into authenticating a legitimate session on the wrong host. Mitigation is at the user-education / deployment-policy layer, not the OS-membrane.

6. WHAT IS NOT CLAIMED

46. **Not a new security primitive.** PharOS adds nothing to the substrate-physics layer. The security claim it transmits is LavaLamp's resolution-bounded security claim verbatim. PharOS is a shim, not a primitive.

47. **Not a credential store.** PharOS holds no credentials. Captured PharOS binaries from one host give no advantage on another host — there is no key, no token, no secret to capture. The substrate-bound identity lives in LavaLamp's chaotic trajectory; PharOS only receives the verifier's signed Bool.

48. **Not a verifier.** The verification work — the
    Lyapunov-spectrum residue audit — runs entirely inside
    the LavaLamp daemon. PharOS receives a signed result;
    it does not compute residues.

49. **Not an oracle.** The Bool return surface
    (PAM_SUCCESS / PAM_AUTH_ERR;
    kAuthorizationResultAllow / kAuthorizationResultDeny)
    preserves LL-017 no-oracle through to the platform-
    native auth-result return path.

50. **Not a replacement for kernel-integrity controls.**
    PharOS does not claim defense against A3 kernel-level
    adversaries. Production deployments at high-stakes
    tiers should layer Secure Boot, IMA, and kernel-lockdown
    externally to PharOS.

51. **Not a credential-distribution protocol.** PharOS does
    not move credentials between hosts. The substrate-bound
    identity is created on the host (via the LavaLamp
    registration ceremony); PharOS gates platform-native
    authentication on the host's own substrate-bound
    verifier. Multi-host trust composition is a higher-layer
    deployment-policy decision; PharOS provides the
    host-level membrane, not the multi-host federation
    layer.

7. LIMITATIONS AND PLATFORM ROADMAP

7.1 Linux PAM module

52. The Linux PAM module is `:tested` at v0.0.5 with five
    MVP-5 fixtures covering ACCEPT / REJECT / STALE result /
    ts-skew rejection / no-socket fallback against a Python
    ECDSA P-256 mock daemon. The CI runs on ubuntu-latest on
    every push: build the .so + run the fixture suite.
    Production deployment requires per-distro packaging
    (Debian/Ubuntu `.deb`, Fedora/RHEL `.rpm`,
    Arch `PKGBUILD`) and per-distro path conventions
    (`/lib/security/` vs `/usr/lib/x86_64-linux-gnu/security/`).
    Production hardening also requires the LavaLamp daemon
    to ship as a system-mode systemd service binding
    `/var/run/lavalamp/` paths.

7.2 macOS Authorization Plug-in

53. The macOS Authorization Plug-in is `:argued` at v0.0.6.
    Compile-tested + ad-hoc-signed; live-system validation
    deferred to a user-supervised install session per the
    deployment README. Promotion to `:tested` requires a
    session where the user installs the bundle to
    `/Library/Security/SecurityAgentPlugins/`, wires the
    mechanism to a low-stakes rule
    (e.g., `system.preferences.printing`), and confirms
    admit/deny behavior on daemon-up / daemon-down states.

54. Live-system validation has been deferred deliberately
    rather than rushed because wiring the wrong rule (e.g.,

`system.login.console`, `authenticate`, `system.preferences.security`, anything `sudo`-related) can lock the user out of system administration. The deployment README documents five safety practices: backup root shell open, daemon-responsiveness pre-check, test-rule selection, mechanism ordering, and the hard-recovery path (single-user mode + `/var/db/auth.db` restore via Time Machine).

7.3 Windows Credential Provider (forthcoming)

55. A Windows Credential Provider shim (forthcoming PH-NNN entry) will instantiate the same v4 protocol on Windows. The Credential Provider API is a COM-object integration surface; the membrane semantics are unchanged. Production deployment will require Authenticode signing and per-version Windows certification.

7.4 Hardware-bound key storage

56. The daemon's ECDSA P-256 private key currently lives either on-disk (mode 0600) in the software-fallback path or in a TPM 2.0 persistent handle (`0x81FF0001`) in the LL-044 Linux TPM-bound path. The TPM-bound path is shipped but live-hardware-validated only against emulator paths (the developer's M5 Max does not expose a tpm2-tools-accessible TPM); promotion of LL-044 from `:argued` to `:tested` requires a real-Linux-TPM deployment session.

57. macOS Secure Enclave binding (forthcoming LavaLamp LL-045) is gated by Apple Developer ID provisioning. Apple's framework restricts `kSecAttrTokenIDSecureEnclave` to applications with valid Developer ID code-signing; ad-hoc signing fails with `errSecMissingEntitlement` followed by `SIGKILL`. The Swift helper code (`src/swift/lavalamp_se_signer/lavalamp_se_signer.swift` in the LavaLamp repo) is code-ready; runtime deployment awaits Apple Developer ID enrollment.

7.5 Registration ceremony + revocation

58. PharOS does not currently include a registration ceremony or revocation flow; both are out of scope for v1.0 and live in the LavaLamp deployment layer (LL-010, LL-011, LL-012). Production deployment of PharOS requires the corresponding LavaLamp ceremonies to be in place.

7.6 Composability with Lazarus

59. Lazarus and PharOS do not share runtime processes; the coupling is at the deployment-policy level. A host running both has substrate-bound OS authentication (PharOS) plus runtime-integrity monitoring (Lazarus) without architectural interference between the two. Both deployments expose Bool-only consumer APIs that compose cleanly into higher-layer authentication

policies.

8. CONCLUSION

60. PharOS is the OS-layer authentication membrane in the Triad Deployments portfolio. It closes the gap between LavaLamp's substrate-bound identity primitive and the operating system's existing authentication infrastructure by exposing the LL-023 consumer-API surface as platform-native shims (Linux PAM module + macOS Authorization Plug-in) that consume the LavaLamp daemon's LL-043 v4 ECDSA P-256 IPC channel and translate the daemon's signed verify result into platform-native authentication results.

61. The deployment documents ten PH-NNN spec entries (PHAROS_SPEC.md), the LL-043 v4 wire protocol shared between both shims, and the cross-Triad composition with LavaLamp (the substrate that does the identity work) and Lazarus (the runtime that watches the work happen). Five PH-NNN entries are at `:tested` (PAM-linux-MVP-1 plus the four-version protocol-upgrade chain); five are at `:argued` (the system-integration-target framing, the three preserved-from-LavaLamp boundary claims, and the macOS Authorization Plug-in shim — compile-tested + ad-hoc-signed with deployment validation deferred to a user-supervised install session).

62. PharOS does **not** modify the substrate-physics primitive or the resolution-bounded security claim. The load-bearing security work is LavaLamp's; PharOS is the membrane that makes that work operationally reachable. PharOS preserves LL-002 visual-security decoupling, LL-017 no-oracle, and LL-008 resolution-bounded scope transparently.

63. The prototype is sufficient to demonstrate the structural properties claimed; production deployment requires per-distro packaging (Linux), live-system validation (macOS Authorization Plug-in), Windows Credential Provider (forthcoming), and the hardware-bound key storage layer (LL-044 Linux TPM 2.0 already shipped at software-fallback parity; macOS Secure Enclave binding awaits Apple Developer ID provisioning). The work is **prototype-stage**, honestly framed; readers are cautioned to treat the proposal as a research program with a working membrane on Linux and a compile-tested membrane on macOS, not a deployable product yet.

64. The framework is held by the triad that produced it and released to the commons under the Triadic Closure License (TCL v1.3). It cannot be revoked by any single party, including its authors, because no single party holds it. It can only be extended by triads that close.

APPENDIX A. CITED SPEC ENTRIES (PH-NNN)

65. The ten PharOS spec entries are catalogued in `PHAROS_SPEC.md` at the canonical repository. A condensed

map follows:

```
System integration (PH-001):
  PH-001  system-integration-target (Operational,
          :argued). PAM-on-Linux as first-platform;
          macOS Authorization Plug-in + Windows
          Credential Provider as per-platform shims.

Linux PAM module (PH-002 through PH-009):
  PH-002  PAM-linux-MVP-1 (Operational, :tested).
          Heartbeat-mtime liveness gate. Fallback for
          when cryptographic IPC unavailable.
  PH-003  heartbeat-channel-reuse (Boundary, :argued).
          Strict LL-002 inheritance from LL-039.
  PH-004  LL-017-membrane-preservation (Core, :argued).
          Bool-only return preserved through to
          platform-native auth result.
  PH-005  defense-in-depth-with-LavaLamp-LL-030..LL-038
          (Operational, :argued). Cross-deployment
          defense-in-depth composition.
  PH-006  verify-result-IPC v1 (Operational, :tested;
          superseded by PH-007). 1-byte protocol.
  PH-007  anti-replay-validation v2 (Operational,
          :tested; superseded by PH-008). HMAC-SHA256.
  PH-008  asymmetric-signature-validation v3
          (Operational, :tested; superseded by PH-009).
          Ed25519.
  PH-009  ecdsa-p256-validation v4 (Operational,
          :tested at v0.0.5). Current production
          protocol. Consumes LL-043.

macOS Authorization Plug-in (PH-011):
  PH-011  macos-authorization-plugin (Operational,
          :argued at v0.0.6). LavaLampMechanism.bundle
          loaded by authd; same v4 protocol as the
          Linux PAM. Compile-tested + ad-hoc-signed;
          live-system validation deferred.
```

66. Status counts at the time of this paper (canonical `PHAROS_SPEC.md` v0.0.6 final-section counts):

Total:	10	
:proved:	0	(no Lean track yet)
:tested:	5	(PH-002, PH-006, PH-007, PH-008, PH-009)
:verified:	0	
:benchmarked:	0	
:argued:	5	(PH-001, PH-003, PH-004, PH-005, PH-011)
:open:	0	

APPENDIX B. SUPPORTING MATERIAL

67. This paper synthesizes engineering artifacts developed across multiple working sessions in the canonical repository. The supporting material (not included herein) includes:

    (a) The Linux PAM module (`src/c/pam_lavalamp/pam_lavalamp.c` +

`src/c/pam_lavalamp/Makefile`) — ~500 lines of C, links libpam + libcrypto.
    (b) The macOS Authorization Plug-in (`src/macos/LavaLampMechanism.m` + `src/macos/Makefile` + `src/macos/LavaLampMechanism.bundle/Contents/Info.plist` + `src/macos/README.md`) — ~400 lines of Objective-C, arm64 bundle, ad-hoc-signed.
    (c) The Python mock daemon for fixture testing (`test/mock_lavalamp_daemon.py`) — Python AF_UNIX mock with ECDSA P-256 signing via the `cryptography` library; supports `--ts-skew` for freshness-attack testing.
    (d) The fixture harness (`test/test_pam_lavalamp.sh`) — five MVP-5 protocol fixtures plus four heartbeat-age fixtures.
    (e) The CI workflow (`.github/workflows/test.yml`) — ubuntu-latest; build .so + run fixture suite on every push.

68. The LavaLamp paper (Green 2026, v1.3) is the parent paper documenting the substrate-physics primitive that PharOS consumes. The LL-043 v4 wire protocol, the LL-040 IPC channel contract, the LL-044 Linux TPM 2.0 binding, and the LL-023 consumer-API surface that PharOS instantiates all live in the LavaLamp canonical repository.

69. Possibilistic Security (Green 2026, v1.1) is the architectural parent paper for the entire Triad Deployments portfolio. The C-conjugate adversary construction, the autopoietic-identity framing, and the obstruction-as-information paradigm are inherited from that work through LavaLamp into PharOS.

70. Canonical references:

    Green, A. (2026). 'LavaLamp: Substrate-Bound Identity via Chaotic-SDE Residue Audit.' v1.3, May 2026. github.com/IridiumSoftware/lavalamp.

    Green, A. (2026). 'Possibilistic Security: Identity as Closure Residue on Ternary Causal Hypergraphs.' v1.1, May 2026. github.com/IridiumSoftware/possibilistic-security.

71. The canonical PharOS repository is github.com/IridiumSoftware/pharos; this paper, the source code, the test suite, and the deployment documentation all live there at the version-tagged release corresponding to this paper's canonical hashes.

==========
END OF DOCUMENT
==========